SYSTEMS NOTE / MARKET INFRASTRUCTURE / 2026-06-19

A cache-resident matching engine for limit order books.

A single-threaded C++17 order book that keeps the entire hot path inside the CPU cache: contiguous price levels instead of trees, a fixed startup arena instead of the allocator, and a lock-free single-producer/single-consumer ring decoupling ingestion from matching. Every number on this page is emitted by the engine itself.

Engine C++17, header-only **SIMD** NEON price scan **Measured on** Apple M4

throughput · replay	median latency	data races	hot-path allocations
2.8M ev/s	**63** ticks	**0**	**0**
decode → route → match, sustained	p50, add/cancel hot path	ThreadSanitizer, 10,000 events	bounded pools, no `new` while matching

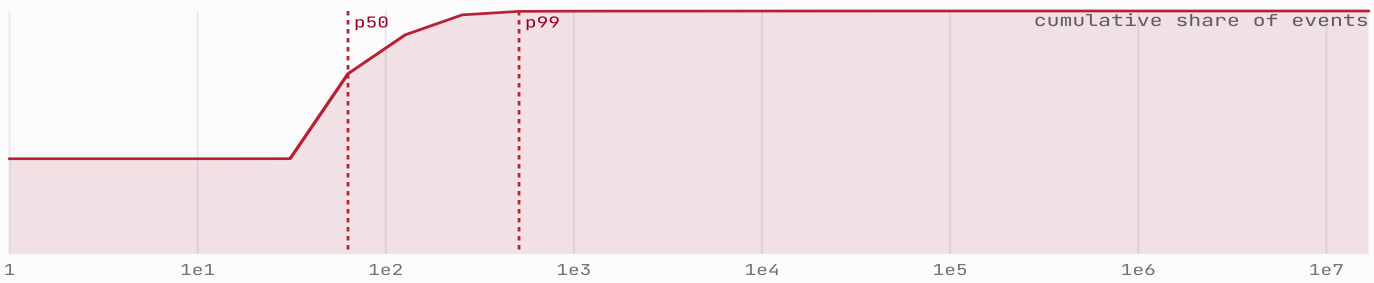


Fig. 1 Cumulative latency distribution over 5,000,000 add/cancel events. Half of all events complete in 63 counter ticks or fewer; the curve reaches the p99 by 511.

01 The feed and its binary protocol

The engine consumes a fixed-width binary feed: every command — limit, market, cancel, modify, replace — is a **40-byte little-endian record**. Fixed width means the decoder never branches on length and the producer never allocates: messages are blitted into the ring as raw bytes and parsed on the far side.

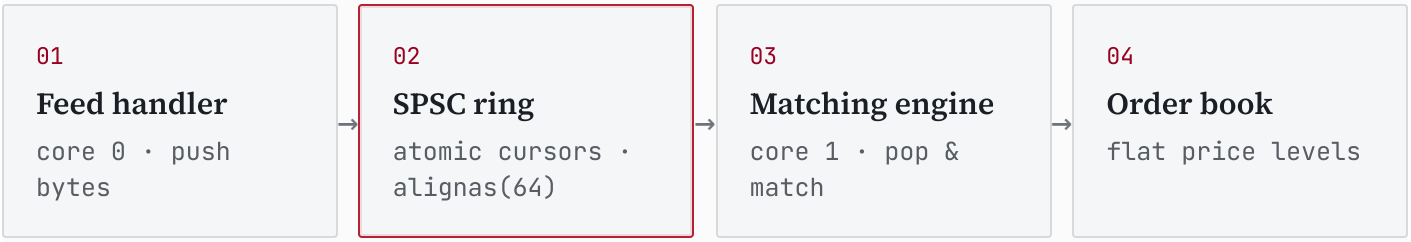
Fig. 2 The wire record. Order-id fields are shaded crimson, the `'OBL1'` magic trailer slate. Decode validates the magic and the type/side ranges before a message is admitted, so a corrupt frame is rejected rather than matched.

A frame is accepted only when its trailing magic word equals `0x4F424C31` (ASCII `"OBL1"`) and its type and side bytes are in range; otherwise it is dropped at the boundary. The replay benchmark on this page round-trips a recorded stream through this exact codec — `1,100,000` encode/decode/match cycles — with **zero** rejects.

02 Lock-free transport

Ingestion and matching run as two stages joined by a single-producer/single-consumer ring buffer. There is no mutex and no condition variable: the only shared state is a pair of 64-bit cursors. The capacity is a power of two, so the slot index is a mask, not a modulo:

$$\text{slot}(i) \;=\; i \bmod N \;=\; i \,\&\, (N-1), \qquad N = 2^{16} = 65{,}536 \text{ slots}$$



Correctness rests on release/acquire ordering rather than sequential consistency. The producer publishes a slot with a `store(tail, release)`; the consumer observes it with a `load(tail, acquire)`. That single edge *synchronizes-with* the read, guaranteeing the buffer write is visible before the slot is dequeued — the minimum barrier the hardware needs, and no more. The ring is **full** when `tail - head = N` and **empty** when `tail = head`; the two cursors sit on separate 64-byte cache lines via `alignas(64)` so the producer and consumer never invalidate each other's line. ThreadSanitizer (§04) exercises this path and reports `0` races.

03 The matching engine and its memory

The book is the part a textbook would get wrong. A `std::map` of price levels is asymptotically tidy and cache-hostile: every node is a pointer chase into cold heap. Here both sides are **fixed-size contiguous arrays** of price levels, sorted by price; orders are drawn from a startup object pool and linked intrusively within each level. A `PriceLevel` is exactly 32 bytes — one half cache line, the width the SIMD scan loads at a time.

Resting orders obey strict **price-time priority**. For two resting bids a, b, a executes first exactly when

$$a \prec b \iff p_a > p_b \ \vee \ \big(p_a = p_b \wedge t_a < t_b\big),$$

with the price comparison reversed for asks. A *modify* that only reduces quantity keeps the order's place in line; a *cancel-replace* deliberately forfeits it. The cumulative depth below is a two-sided book built and matched entirely by the engine, then snapshotted:

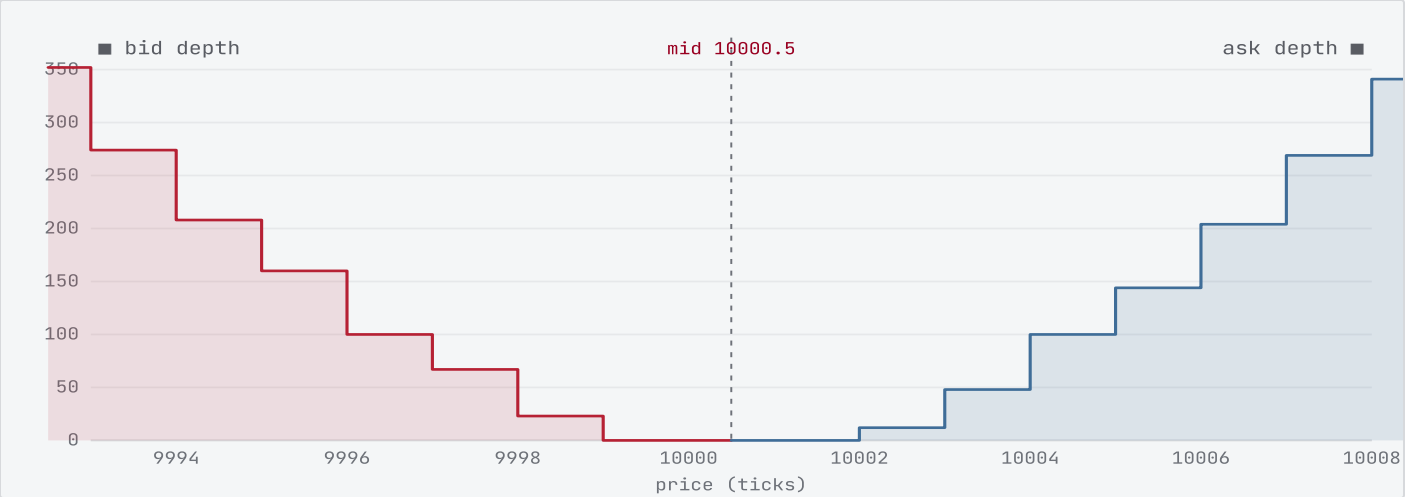


Fig. 3 Cumulative depth after two crossing market orders consume the touch. Best bid 9999, best ask 10002, a 3-tick spread. The staircase is the engine's own `bid_depth()/ask_depth()` snapshot — internal storage is never exposed.

The crossing prints a deterministic trade tape:

SEQ	AGGRESSOR	PRICE	QTY
#1	BUY	10,001	8
#2	BUY	10,001	8
#3	BUY	10,002	12
#4	SELL	10,000	10
#5	SELL	10,000	10

SEQ	AGGRESSOR	PRICE	QTY
#6	SELL	9,999	4

Tbl. 1 The 6 trades generated by the snapshot scenario, each tagged with the aggressor side, price, and filled quantity.

Because every container is fixed at construction, the engine's working set is bounded and known ahead of time:

$$M \;=\; N_o\,\mathrm{sizeof}(\mathrm{Order}) \;+\; L\,\mathrm{sizeof}(\mathrm{PriceLevel}) \;+\; C\,\mathrm{sizeof}(\mathrm{Entry}),$$

sizeof PriceLevel	*sizeof Order*	*ring capacity*	*book arena*
32 B	40 B	65,536	8.6 MB
one 256-bit SIMD / half-cache-line record	pooled, intrusively linked	power-of-two SPSC slots	resident, allocated once

When a marketable order arrives, the engine scans the price array with a vector compare — AVX2 on x86, NEON on Apple Silicon, scalar elsewhere — testing eight (or four) levels per instruction and reading the first crossing index from the comparison mask. The short, hot array stays in L1; the scan finishes before a tree's first pointer dereference would have returned from memory.

04 Correctness and race-freedom

Speed claims are only worth as much as the correctness underneath them. The engine ships with a deterministic test suite, a golden-file replay, and a sanitizer pass over the threaded pipeline. Every check below runs in the build that produced this page:

CHECK	COVERS	RESULT
● order_book_tests	Matching, partial/full fills, price-time priority, modify, replace, depth, SPSC FIFO	PASS
● protocol_tests	40-byte binary message encode / decode round-trip	PASS
● replay_golden	Deterministic replay matches the committed golden output byte-for-byte	PASS

Tbl. 2 Verification matrix. A filled crimson dot marks a passing check; the status is also spelled out, so the table never relies on color alone.

The threaded pipeline is then rebuilt with `-fsanitize=thread` and driven with `10,000` rapid-fire events across the producer, ring, and consumer. ThreadSanitizer reports `0` data races — empirical backing for the release/acquire reasoning in §02, not a substitute for it.

05 Latency and throughput

Latency is timed per event with the hardware cycle counter — `__rdtsc()` on x86, the virtual counter on ARM — and bucketed into a power-of-two histogram, where bucket b collects every event whose tick count falls in its octave:

$$b(x) = \lfloor \log_2 x \rfloor, \qquad \text{bucket } b \text{ covers } \left(2^{b},\, 2^{b+1}\right].$$

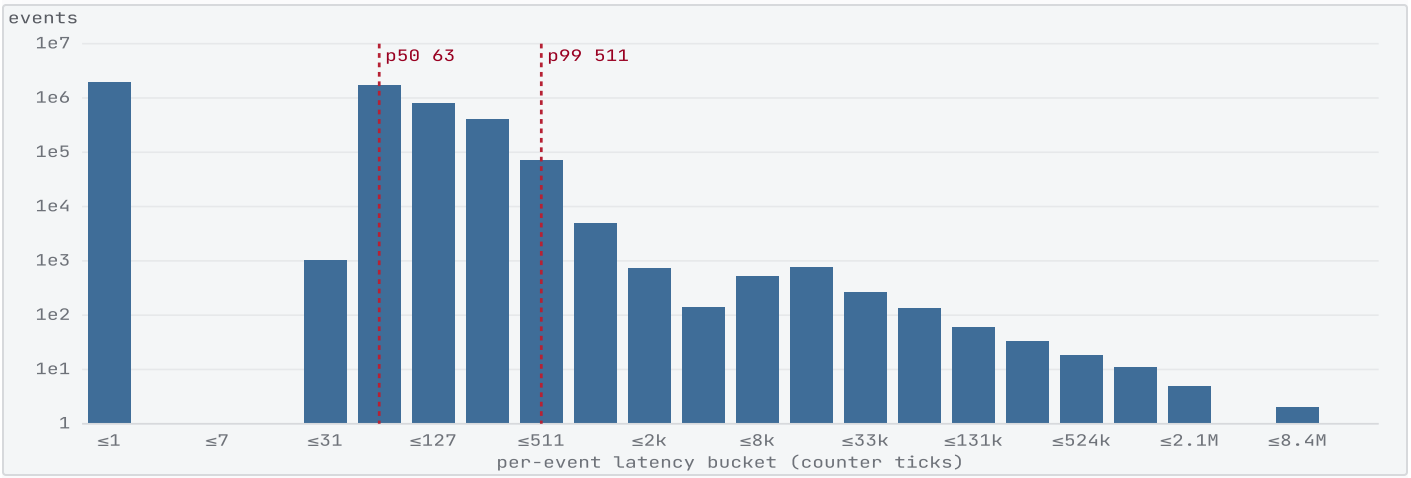


Fig. 4 Per-event add/cancel latency, counts on a log axis. The mass sits in the low-tick octaves — p50 at 63, p99 at 511 ticks — with a thin tail from scheduler interrupts the single thread cannot mask.

Plotting the tail by percentile shows the same story for both the bare hot path and the full decode→route→match replay: flat through the body, with the climb deferred deep into the nines.

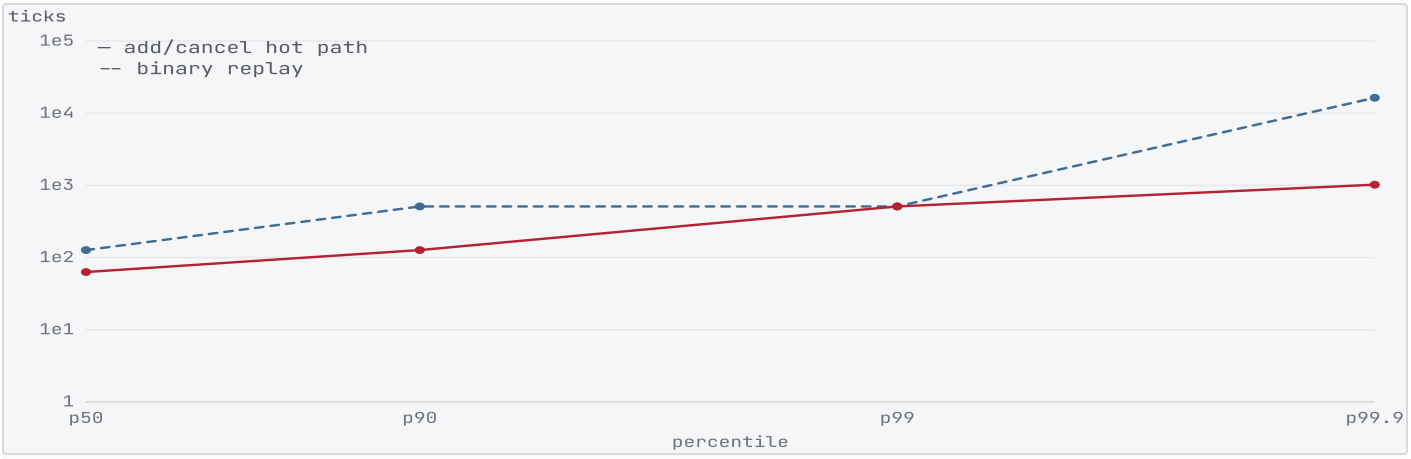


Fig. 5 Tail latency by percentile (log ticks). The replay path (dashed slate) carries decode and symbol routing on top of matching, so it sits above the bare hot path (crimson) — both stay bounded to the p99.9.

2.8M

events / second, decode → route → match

The design target was 2 million events per second on one thread. Every workload clears it: the single-thread paths by several times over, the two-thread pipeline — which pays for a cross-core handoff — by the median of its size sweep. All figures are medians of repeated runs on a loaded laptop, so pinned, isolated production cores would read higher.

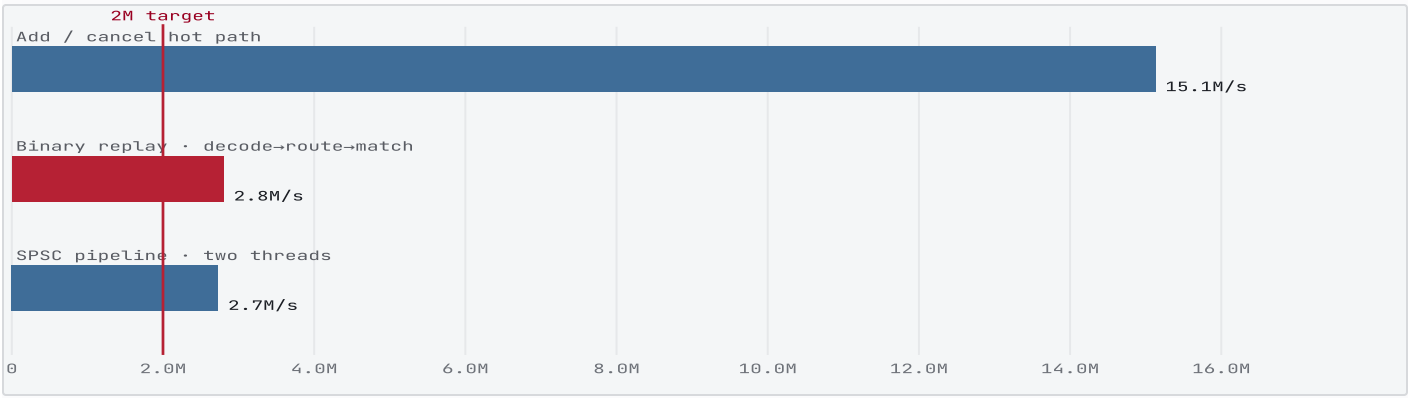


Fig. 6 Throughput by workload against the 2M/s target (crimson). The replay bar (accent) is the headline figure: a full binary pipeline, not an isolated micro-benchmark. The pipeline bar is the median of the size sweep in Fig. 7.

Throughput holds as the workload grows; smaller runs read higher because the working set stays warm in cache, which is itself the point of the flat layout.

Fig. 7 SPSC pipeline throughput versus run size (log). The engine sustains multi-million-event-per-second matching across two orders of magnitude of batch size.

How to read the latency numbers. The counter unit is architecture-specific — ARM's virtual counter ticks slower than an x86 TSC cycle — so the ticks here are honest relative magnitudes, not nanoseconds.

For a nanosecond claim, pin threads to isolated cores, disable frequency scaling, and convert ticks with the measured invariant counter frequency, as `PERFORMANCE.md` describes.

Colophon

The engine is header-only C++17 — flat price-level arrays, a startup object pool, an open-addressed order index, a lock-free SPSC ring, and SIMD price scans — built with a plain Makefile. This note is a single HTML page printed to PDF through headless Chrome; every figure is generated by `site_export`, which runs the real engine and emits the `data.json` the page binds to. No number is hand-written.

Built from `fa13d62` · `Apple clang version 17.0.0 (clang-1700.3.19.1)` · generated `2026-06-19T20:54Z`.

```
# reproduce every figure
make test      # correctness
make tsan      # 0 data races
make site      # engine → JSON

PYTHONPATH=src \
  python3 site/export_data.py
bash site/build_pdf.sh
```